

```
}  
}
```

Note - There are 3 String variables referenced here and 5 blocks of character storage are allocated?

```
public class Test  
{  
    public static void main( String[] args )  
    {  
        String w = "Welcome to ";  
        String j = "Java ";  
        float v = 1.2f;  
        String txt;  
        txt = w + j + v + ".";  
        System.out.println( txt );  
    }  
}
```

Why does such a tiny program need so many areas of memory? Let's explore the answer to that question in detail. All Java characters are stored in 16-bit Unicode, so they are able to store international character sets. In Unicode, every character requires 2 bytes. Under this system, "Hello, world." would require 26 bytes to store in memory.

```
To create Strings, we do something like,  
String myname = "Joe";  
//initialized to Joe  
or  
String myname = new String("Joe" );  
// initialized to Joe
```

The bottom method is less familiar, but more correct under the Object-Oriented method. The top method is a convenience we use in order to make String manipulation easier. Another convenience is in String concatenation:

```
String a = "hello ";  
String b = "world";  
String c;
```